

TP : Communication, Sécurité et Contrôle d'un Robot Mobile sous ROS 2

Filière Robotique

Mai 2026

Table des matières

1	Partie 1 : Initialisation du Système (Vision par Moments & Contrôle P)	2
1.1	Idée générale	2
1.2	Travail demandé	2
2	Partie 2 : Optimisation Dynamique (Correcteur PD & Machine à États)	3
2.1	Idée générale	3
2.2	Travail demandé	3

Travaux Pratiques : Robotique Mobile & Autonomie

Partie II : Guidage par Vision Artificielle et Intégration Système sous ROS 2

Description du TP

L'objectif de ce TP est de mettre en œuvre un asservissement visuel rudimentaire. Dans une première partie, vous connecterez un nœud de traitement d'images OpenCV basé sur la méthode des **Moments géométriques** à un contrôleur proportionnel (P). Dans une seconde partie, vous ferez évoluer ce contrôleur vers un correcteur **Proportionnel-Dérivé (PD)** supervisé par une **Machine à États Finis (FSM)** afin de détecter et négocier les virages en épingle ainsi que les pertes de piste.

1 Partie 1 : Initialisation du Système (Vision par Moments & Contrôle P)

1.1 Idée générale

Le nœud de vision extrait une *Région d'Intérêt (ROI)* en bas de l'image pour anticiper la trajectoire immédiate du robot. À l'aide d'un seuillage et du calcul des moments d'ordre 0 et 1, on extrait la coordonnée horizontale C_x du centre de la ligne pour en déduire l'erreur de centrage par rapport à l'axe optique de la caméra.

1.2 Travail demandé

1. **Analyse et correction du binaire (Code Vision)** : Regardez la ligne du code de vision :

```
_, thresh = cv2.threshold(roi_gray, 200, 255, cv2.THRESH_BINARY)
```

Sachant que la piste est constituée de **lignes noires sur un sol clair**, expliquez pourquoi l'application de `cv2.THRESH_BINARY` va fausser le calcul du centre de la ligne opéré par `cv2.moments(thresh)`. Quelle modification devez-vous apporter au drapeau (*flag*) de la fonction pour corriger ce problème ?

2. **Compréhension mathématique des moments** : Rappelez la formule mathématique des moments M_{00} et M_{10} pour une image binarisée. Que représente physiquement le rapport $\frac{M_{10}}{M_{00}}$ calculé dans le code ?
3. **Mise en réseau ROS 2** : Lancez le nœud de vision corrigé ainsi que le nœud de commande proportionnel (RobotSimple - Code 2). Observez le comportement du robot. Pourquoi le déplacement linéaire est-il instable et saccadé lorsque le robot commence à osciller ?

2 Partie 2 : Optimisation Dynamique (Correcteur PD & Machine à États)

2.1 Idée générale

Pour lisser la trajectoire, nous basculons sur un correcteur **Proportionnel-Dérivé**. De plus, pour éviter que le robot ne rate la piste lors d'un virage brusque, nous implémentons une logique comportementale sous forme de Machine à États :

- **LIGNE_DROITE** : Erreur faible → Vitesse linéaire nominale élevée, correction douce.
- **VIRAGE_BRUSQUE** : L'erreur dépasse un seuil critique → On divise la vitesse linéaire par deux pour donner au robot le temps de pivoter.
- **DEMI_TOUR** : Si l'erreur devient maximale ou si la ligne sort brusquement du champ → Le robot applique un pivot sur place sécurisé pour retrouver la piste.

2.2 Travail demandé

1. **Analyse des bugs du Code 3** : Le script PD fourni par l'enseignant comporte des erreurs de syntaxe Python provoquant un crash au démarrage (problèmes de portée de variables et d'imports). Identifiez-les et corrigez-les.
2. **Rôle du terme Dérivé** : Lors de l'entrée dans une courbe, comment réagit le terme dérivé par rapport au terme proportionnel ? En quoi cela aide-t-il à anticiper le virage ?
3. **Implémentation de la FSM (Code à trous)** : Complétez le script de la page suivante pour intégrer la machine à états décrite dans le diagramme. Testez votre code sur la piste et validez le comportement lors des virages à 90 degrés.